

QUANT · Created Thu, May 21, 2026 · Updated Thu, May 21, 2026

Walk-Forward Optimization: Anchored vs. Rolling Windows and When Each Fails

Walk-forward validation is the closest thing I have to a non-negotiable in my [validation pipeline](#). Every strategy that makes it past exploratory analysis and statistical testing has to survive walk-forward before I take it seriously. The idea is simple: train on the past, test on the future, slide forward, repeat. But the implementation decisions (which window type, how long, how much gap between training and testing) are where most of the real work happens and where most of the mistakes hide.

I've been rebuilding my quant workflow since returning to quantitative work after fourteen years in production software engineering, and walk-forward is the piece I spent the most time getting right. This article covers how I set up walk-forward testing, the choices I've made and why, and the problems I've run into.

WHAT WALK-FORWARD VALIDATION IS

The procedure is straightforward:

1. Define a training window and a test window.
2. Train or optimize the strategy on the training window.
3. Test the strategy on the test window without re-optimizing.
4. Slide forward by one test window.
5. Repeat until you run out of data.
6. Concatenate the test-window results for the OOO (out-of-sample) performance estimate.

The critical constraint is that the strategy never sees future data during training. Every test window contains only data that comes after the training window. This is what separates walk-forward from standard cross-validation approaches that shuffle data randomly and violate the arrow of time.

The concatenated OOS results give you something closer to what the strategy would have experienced if you'd actually been trading it, re-calibrating at each step using only the information available at the time.

ANCHORED VS. ROLLING WINDOWS

This is the first real decision, and it shapes everything downstream.

Anchored (expanding) windows start the training period at the beginning of your data and grow it with each step. The first iteration trains on years 1-2 and tests on year 3. The next trains on years 1-3 and tests on year 4. The training set gets larger with each step.

I reach for anchored windows when the strategy benefits from long histories. Mean-reversion strategies and statistical arbitrage setups fall into this category. If the edge comes from a structural relationship that's stable over time, more data means a better estimate of that relationship. Anchored windows also make sense when regime changes are infrequent enough that old data is still informative.

The downside is that early data can dominate. If the market microstructure shifted significantly partway through your sample, the anchored window drags along data from a regime that no longer applies, diluting the estimate with irrelevant history.

Rolling windows keep the training period at a fixed length and slide it forward. Train on years 1-2, test on year 3. Then train on years 2-3, test on year 4. Old data falls off the back as new data enters.

I use rolling windows when markets are non-stationary or the strategy adapts to recent conditions. ~~FX~~ markets in particular go through regime shifts (changes in central bank policy, volatility regimes, correlation structures) where data from three

years ago may actively mislead. Rolling windows naturally adapt by forgetting stale data.

The cost is that you’re always working with less training data than the anchored approach would give you. Fewer observations mean noisier parameter estimates, which means noisier OOS performance.

Here’s how both approaches look sliding through a five-year dataset:

```
Anchored (expanding):
Step 1: [===TRAIN===][TEST].....
Step 2: [====TRAIN====][TEST].....
Step 3: [=====TRAIN=====][TEST]...
Step 4: [=====TRAIN=====][TEST]

Rolling (fixed):
Step 1: [===TRAIN===][TEST].....
Step 2: ....[===TRAIN===][TEST].....
Step 3: .....[===TRAIN===][TEST].....
Step 4: .....[===TRAIN===][TEST]..
```

	ANCHORED	ROLLING
When to use	Stable structural relationships, infrequent regime changes	Non-stationary markets, strategy adapts to recent conditions
Training data	Grows with each step	Fixed size
Pro	Maximum use of available data	Naturally forgets stale regimes
Con	Early data can dominate, diluting recent signal	Less data per window, noisier estimates
Example	Stat arb on a persistent spread	Mean reversion on FX pairs during policy shifts

In practice, I run both. If a strategy passes walk-forward under both anchored and rolling configurations, that’s meaningful. If it only passes under one, I want to understand why. A strategy that needs the full history to work may be capturing a real long-term relationship. A strategy that only works with rolling windows may be

adapting to short-term regimes. Either can be valid, but they imply different things about what the strategy is actually doing.

THE META-PARAMETER PROBLEM

Walk-forward validation has its own parameters: training window length, test window length, and step size. These are meta-parameters. If you try ten different window length combinations and pick the one that gives the best OOS Sharpe, you've overfit the validation procedure itself.

This is second-order overfitting. First order is optimizing strategy parameters to fit historical data. Second order is optimizing the validation setup to make the strategy look good. The second is harder to catch because it feels like you're being rigorous. You're doing walk-forward, after all.

My approach is to commit to window sizes based on the market microstructure I'm trading before I see any results. If I'm testing a mean-reversion strategy on FX pairs where the typical reversion period is 5-15 trading days, the training window should be long enough to estimate that reversion reliably (I've found that at least 100 reversion cycles gives stable estimates, so roughly 2 years of daily data for a signal with a 5-day half-life) and the test window should contain enough trades to be statistically meaningful (at least 20-30 trades, so 1-3 months depending on signal frequency).

I document the reasoning for these choices before running anything. The documentation is the discipline. If I can't articulate why a particular window length makes sense for the strategy I'm testing, that's a sign I don't understand the strategy well enough to validate it.

After the primary walk-forward run, I do check robustness by running the same strategy through two or three alternative window configurations. But I'm looking for consistency, not optimization. If the strategy passes at 2yr/3mo, 3yr/6mo, and 18mo/2mo, it's robust. If it only passes at exactly 2yr/3mo, the result is fragile and I don't trust it regardless of how good the numbers look. Pardo covers this

methodology thoroughly in [*The Evaluation and Optimization of Trading Strategies*](#) , which remains the most practical reference on walk-forward parameter selection.

PURGING AND EMBARGO: HANDLING LEAKAGE AT BOUNDARIES

The subtlest problem in walk-forward validation is leakage at the boundary between training and test windows. The last observation in your training data and the first observation in your test data are adjacent in time. If returns are autocorrelated, if your features use rolling windows, or if your labels look forward, information bleeds across that boundary.

Purging removes a buffer of observations between the training and test windows. The purge width should be at least as long as the [autocorrelation](#) decay in your data. For daily FX returns with significant autocorrelation at lag 5, I purge at least 5 observations. For strategies that use features with a 20-day rolling lookahead, the purge needs to be at least 20 observations, because the first test observation's features reach back 20 days into the training period. Those shared input observations mean the test window isn't truly independent of training. The model was fit on data that also feeds into the test features.

Embargo is conceptually similar but applied to the test side: you don't evaluate the first K observations of the test window. This handles the case where the strategy's initial trades in a new test window are still influenced by patterns that were present at the end of training.

The forward-looking label problem is the easiest to get wrong. If your strategy uses a label like "will the price be higher in 5 days?", the training set's last observation has a label that looks 5 days into the future, right into the start of the test window. The purge width must be at least as long as the label horizon. I've caught this in my own code more than once.

The cost of purging is that you lose data. A 20-day purge on each walk-forward step adds up quickly. On a three-year daily dataset, even a handful of steps with 20-day purges can throw away several months of observations that never appear in either

training or testing. This is worth it. The alternative is leakage that inflates your OOS estimates, which is worse than a smaller sample.

COMBINATORIAL PURGED CROSS-VALIDATION

López de Prado introduced Combinatorial Purged Cross-Validation (CPCV) in [*Advances in Financial Machine Learning*](#) as a more rigorous alternative to standard walk-forward. The idea: partition the data into N temporally ordered groups, choose K groups for testing, use the remaining $N-K$ for training, apply purging at every boundary, and repeat for all $C(N,K)$ combinations.

Unlike standard walk-forward, which produces a single OOS equity curve, CPCV produces $C(N,K)$ curves. Each is a valid temporal backtest. The distribution of performance across those curves tells you something that a single walk-forward run can't: how stable the strategy's OOS performance is across different temporal splits.

The interpretation is useful. If the median OOS Sharpe across all paths is positive but many individual paths are negative, the strategy works in some regimes and not others. That's important information that a single walk-forward run might hide if it happened to land in favorable regimes.

I don't use CPCV in my regular workflow. The computational cost is the problem. With $N=10$ groups and $K=2$ test groups, you get 45 paths, which is manageable. But $N=20$ and $K=5$ gives 15,504 paths, each requiring a full strategy refit. For strategies that take more than a few seconds to train, this becomes impractical quickly. The paths are independent, so parallelization helps, but it's still a lot of compute for a validation step.

Where I think CPCV earns its cost is for final validation of a strategy you're about to deploy, not during iterative development. You've already used standard walk-forward to narrow the field. CPCV is the expensive, thorough check you run once on the survivor.

STRATEGY SELECTION AND COMMON MISTAKES

Walk-forward is where most strategies die in my pipeline, and that's by design. But the way you use it matters as much as the results.

Define walk-forward parameters before testing any strategy. If you have N candidate strategies, run all of them through the same walk-forward splits. This is the equivalent of using the same test harness for all your code. It ensures the comparison is fair. Choosing different window lengths for different strategies until each one looks good defeats the entire purpose.

Don't re-run walk-forward after tweaking the strategy. This is the most common mistake I see, and I've made it myself. You run walk-forward, see disappointing OOS performance, adjust the strategy, and re-run. The OOS results are no longer truly out-of-sample because you've adapted the strategy based on them. Each iteration burns your holdout data. If you need to iterate, split your data into a development set (for walk-forward during research) and a true holdout (for final evaluation) that you touch exactly once.

Watch the ratio of parameters to training observations. If your strategy has 20 tunable parameters and the training window contains 500 observations, you're almost certainly overfitting in-sample. The training window performance will look great; the OOS will be terrible. A common heuristic for avoiding overfitting to historical noise is to keep the number of free parameters below the square root of the training window length. For a 500-observation window, that means fewer than roughly 22 parameters. For most of my work, I aim for single-digit parameter counts.

Include transaction costs in both training and OOS evaluation. I apply the same cost model to both windows. The strategy optimizes for net returns during training, and performance is reported net during testing. If you optimize gross in training but evaluate net in testing, you're letting the optimizer find parameters that generate excessive turnover it would never choose if it had to pay for it. A strategy that looks good gross but marginal net is not a strategy worth deploying.

Report your methodology. Always document: the number of walk-forward steps, training and test window lengths, purge and embargo lengths, whether you used anchored or rolling windows, and whether the results are robust to alternative

window configurations. If you can't fully describe your walk-forward setup in a paragraph, it's probably too complicated.

WHAT I'VE LEARNED

The thing about walk-forward that took me the longest to internalize is that it's not a rubber stamp. Running walk-forward and getting a positive OOS Sharpe doesn't mean the strategy works. It means the strategy worked in that particular walk-forward configuration, on that particular dataset, with those particular window choices. That's evidence, not proof.

The real value of walk-forward is in what it kills. A strategy that fails walk-forward under multiple window configurations is almost certainly overfit. That's a clear signal. A strategy that passes is a candidate for further testing ([bootstrap stress testing](#) , [bias auditing](#) , parameter sensitivity analysis), not a candidate for deployment. Walk-forward sits in the middle of the [validation funnel](#) , and the funnel only works if each stage is honest about what it can and can't tell you.

The discipline of fixing window lengths before seeing results is something I carried over from software engineering, where you define acceptance criteria before you write the code, not after you see the output. The principle is the same: decide what success looks like before you have a reason to move the goalposts. Documenting that reasoning before running anything is the cheapest form of intellectual honesty in quantitative research.



Susan Potter
Quant

Work with me

I spent the first half of my career building risk models and market data infrastructure at BNP Paribas, Bank of America, and Citadel, then fourteen years shipping production systems at scale. Now I bring both sides to quantitative trading. If you're a trading firm, family office, or fund looking to tighten the connection between your research ideas and your production trading systems, whether that's building validation pipelines, formalizing signal logic, or getting microstructure analytics into a deployable state, I'd like to hear what you're working on. Reach me at me@susanpotter.net.



QUANT
Susan Potter

TAGS

Backtesting Statistics Quantitative Finance Strategy Validation Econometrics

SHARE



FOLLOW



SUBSCRIBE



© 2006 - 2026 Susan Potter · @me@susanpotter.net



[About](#) [Contact](#) [Privacy](#) [Terms](#) [Credits](#)